

QuizMaster Flask App

Project Report

Name: Priyanshi Kwatra

Student Roll Number: 22f1000412

Student Email address: 22f1000412@ds.study.iitm.ac.in

Course: MAD 2 Project

Institution: IIT MADRAS

1. Project Overview

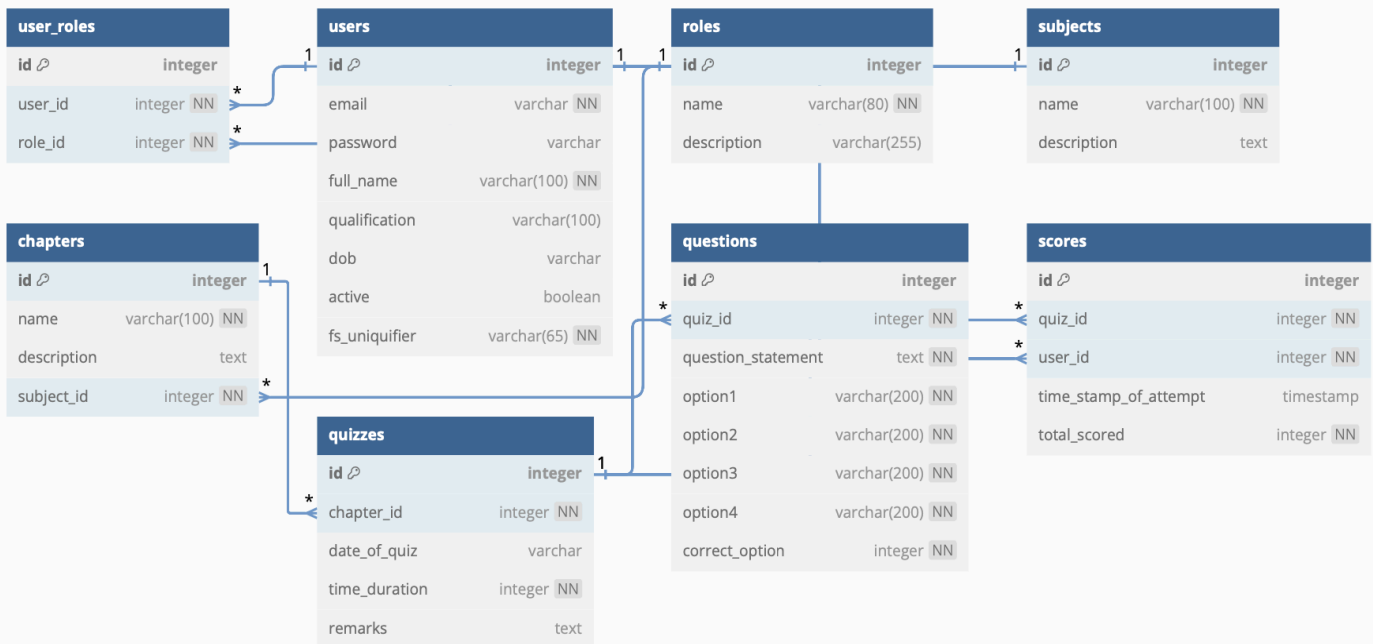
- **Project Name:** QuizMaster Flask App
- **Objective:** The QuizMaster Flask app is an interactive platform where an administrator (quiz master) manages subjects, chapters, and quizzes, while users(Students) can register, log in, and attempt quizzes. The goal is to ensure a smooth and engaging quiz experience with secure authentication, efficient quiz management, and optimized performance.
- **Implementation Approach:**
 - Designed an SQLite-based relational database schema to store users, roles, subjects, chapters, quizzes, questions, and scores.
 - Integrated Flask-Security for authentication and role-based access control.
 - Developed RESTful APIs using Flask to manage quiz-related operations.
 - Implemented Redis for caching and Celery for handling background tasks.
 - Automated scheduled tasks for reminders and monthly reports using Celery.

2. Technologies and Tools Used

- **Backend:** Flask, Flask-SQLAlchemy, Flask-Security, Flask-RESTful
- **Database:** SQLite (as per project guidelines)
- **Caching Mechanism:** Redis
- **Task Management:** Celery
- **Frontend:** Vue.js (as per project requirements)
- **Styling:** Bootstrap (as mandated, without additional CSS frameworks)

3. Database Schema and Relationships

- **Key Tables:**
 - **User** (Stores user details, with a unique ID as the primary key)
 - **Role** (Defines user roles: Admin or User)
 - **UserRoles** (Associates users with specific roles)
 - **Subject** (Stores different subjects)
 - **Chapter** (Belongs to a subject and stores relevant chapter details)
 - **Quiz** (Belongs to a chapter and contains quiz-related information)
 - **Question** (Linked to a quiz and contains multiple-choice questions)
 - **Score** (Tracks users' quiz attempts and performance)



4. Features and Functionalities

- **Admin Capabilities:**
 - Manage subjects, chapters, and quizzes.
 - Add, edit, or delete quiz questions.
 - Monitor user activities
- **User Capabilities:**
 - Register, log in, and attempt quizzes.
 - View past scores and track performance history.
 - Search for existing subjects.
 - Receive email reminders for upcoming quizzes.

5. API Endpoints

- **POST /login** - Authenticate users and admins.
- **POST /register** - Allow users to sign up.
- **GET /subjects** - Retrieve all available subjects.
- **POST /subjects** - Add a new subject (Admin only).
- **GET /quizzes** - Fetch all quizzes.
- **POST /quizzes** - Create a new quiz (Admin only).
- **GET /quizzes/<id>** - Get details of a specific quiz.

- **POST /quizzes/<id>/attempt** - Submit quiz responses.
- **GET /scores** - Retrieve quiz scores.

(Additional API endpoints exist for further subjects, quizzes and user management features.)

6. Background and Scheduled Jobs

- **Daily Reminder Task:**
 - Identifies inactive users and new quiz availability.
 - Send reminders via email (Mailhog).
- **Monthly Report Generation:**
 - Summarizes user activity and quiz performance.
 - Sends an HTML report via email (Mailhog) on the first of each month.

7. Performance Enhancements

- Cached frequently accessed data using Redis to reduce API response time.
- Applied cache expiration policies to ensure data freshness.
- Optimized database queries to improve system efficiency.

8. Frontend Structure and Implementation

- The frontend is developed using Vue.js and follows a modular structure:
 - **components/:** Contains reusable components like `Navbar.js`, `ChartComponent.js`, `SearchComponent.js`.
 - **pages/:** Includes different views such as `Login.js`, `Signup.js`, `DashboardAdmin.js`, `DashboardStud.js`, `AdminSummary.js` etc.
 - **utils/:** Manages routing (`router.js`) and state management (`store.js`).
 - **app.js:** The main entry point for the Vue application.
 - **styles.css:** Contains global styling rules.
 - **templates/:** Houses `index.html`, which serves as the main entry file for the frontend.

9. Deployment and Demonstration

- Runs locally for demo purposes.
- Automatically initializes the database with a default admin and student account.

10. Presentation Video Link

https://drive.google.com/file/d/16lEnGQjsn3z1htdVo_mrGvuakkWnG83g/view?usp=sharing